



Informe de resultados

Tercer cuatrimestre

Aprendizaje no Supervisado

Dr. Ernesto Garcia Amaro

Jessica Melani Romero Lora

24 de mayo del 2026

Tabla de contenido

1. Introducción.....	3
2. Desarrollo de la práctica.....	4
Optimización de la rutina de procesamiento.....	4
2.2 Actividad 2 — Generación de archivos CSV por clase con núcleo dedicado.....	5
2.3 Actividad 3 — Comparación secuencial vs. paralelo.....	7
2.4 Actividad 4 — Tiempo de procesamiento por clase individual.....	8
3.1 Optimización de la rutina de procesamiento.....	10
3.2 Generación de CSV por clase con núcleo dedicado.....	11
3.3 Impacto del paralelismo: secuencial vs. paralelo.....	11
3.4 Análisis del tiempo por clase y desbalance de carga.....	11
3.1 Optimización de la rutina de procesamiento.....	12
3.2 Generación de CSV por clase con núcleo dedicado.....	12
3.3 Impacto del paralelismo: secuencial vs. paralelo.....	12
3.4 Análisis del tiempo por clase y desbalance de carga.....	13

1. Introducción

El procesamiento masivo de imágenes en tareas de aprendizaje automático impone restricciones de rendimiento que no pueden ignorarse cuando el volumen de datos escala hacia miles o millones de muestras. En el contexto de la materia Cómputo de Alto Desempeño, la presente práctica parte de una rutina base de preprocesamiento y aplica cuatro estrategias concretas de optimización orientadas a reducir el tiempo de cómputo mediante el uso eficiente de los recursos del sistema.

El punto de partida es un pipeline de preprocesamiento en Python que, en su versión original, opera de manera secuencial e itera píxel a píxel mediante ciclos anidados para extraer los valores de cada imagen. Aunque funcionalmente correcto, este enfoque no aprovecha las capacidades vectorizadas de las bibliotecas numéricas ni la concurrencia disponible en procesadores multinúcleo modernos.

La práctica aborda cuatro dimensiones de optimización articuladas entre sí: (1) la sustitución de operaciones interpretadas en Python por operaciones vectorizadas compiladas en C a través de NumPy; (2) el diseño de una arquitectura de escritura sin contención, en la que cada proceso trabaja sobre su propio archivo de salida; (3) la medición empírica y comparación del tiempo de ejecución en modalidad secuencial y paralela; y (4) el análisis del tiempo por clase individual como indicador del desbalance de carga entre procesos.

Los resultados obtenidos permiten cuantificar el speedup real alcanzado y, al mismo tiempo, identificar el cuello de botella estructural que limita la escalabilidad del sistema: la asignación estática de una clase completa por núcleo en presencia de un dataset desequilibrado. Ambas observaciones —el beneficio medido y la limitación diagnosticada— constituyen el fundamento empírico de las conclusiones.

El siguiente cuadro sintetiza las cuatro actividades y las técnicas de optimización aplicadas:

#	Actividad	Técnica aplicada
1	Optimización de la rutina de procesamiento	Vectorización NumPy (.flatten()) + lectura directa en gris
2	Generación de un CSV por clase	Proceso dedicado por clase; eliminación de contención en escritura
3	Comparación secuencial vs paralelo	Medición de tiempo con time.perf_counter() y speedup real

4	Tiempo de procesamiento por clase	Queue para comunicación entre procesos; análisis de desbalance de carga
---	-----------------------------------	---

2. Desarrollo de la práctica

Optimización de la rutina de procesamiento

La rutina original extraía el valor de cada píxel mediante dos ciclos for anidados, produciendo una iteración por píxel dentro del intérprete Python. Para una imagen de 16×16 píxeles esto implica 256 iteraciones individuales; con un dataset de aproximadamente 3,663 imágenes, la extracción por sí sola acumula más de 900,000 iteraciones interpretadas.

Importación de librerías:

```
import time
from multiprocessing import Process, Queue, cpu_count
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# --- Rutas ---
DATASET_ROOT = Path("/home/jessi/masters/chapter3//Dataset")
SALIDA_DIR = Path("output")
SALIDA_DIR.mkdir(exist_ok=True)

# --- Parámetros del preprocesamiento ---
CLASES = ["A", "B", "C", "D"]
TARGET_SIZE = (16, 16)
N_PIXELES = TARGET_SIZE[0] * TARGET_SIZE[1]

print(f"Dataset : {DATASET_ROOT}")
print(f"Salida : {SALIDA_DIR.resolve()}")
print(f"Núcleos disponibles: {cpu_count()}")
```

El fragmento original que se reemplazó tenía la siguiente estructura:

```
# Original - O(filas × columnas) iteraciones en Python puro
for y in range(filas):
    for x in range(columnas):
        pix = re_size[y, x]
        file.write(f"{pix},")
```

La solución optimizada aplica tres cambios en cascada:

```
def procesar_imagen(ruta, target_size=TARGET_SIZE):  
    img = cv2.imread(str(ruta), cv2.IMREAD_GRAYSCALE) # lectura directa en gris  
    if img is None:  
        return None  
    resized = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)  
    return resized.flatten() # vectorización - sustituye los ciclos anidados
```

En el notebook mantiene la siguiente estructura:

```
def procesar_imagen(ruta, target_size=TARGET_SIZE):  
    img = cv2.imread(str(ruta), cv2.IMREAD_GRAYSCALE)  
    if img is None:  
        return None  
    resized = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)  
    return resized.flatten() # ← reemplaza for y in filas: for x in columnas  
  
def procesar_clase(clase, dataset_root, salida_dir, queue=None):  
  
    inicio = time.perf_counter()  
  
    rutas = sorted((Path(str(dataset_root)) / clase).glob("*.JPG"))  
    filas = []  
    for ruta in rutas:  
        vector = procesar_imagen(ruta)  
        if vector is not None:  
            # Agrega la etiqueta de clase al final del vector de píxeles  
            filas.append(np.append(vector, clase))  
  
    # Escribe todo en un solo paso: más eficiente que abrir/cerrar por imagen  
    col_nombres = [f"p{i}" for i in range(N_PIXELS)] + ["clase"]  
    df = pd.DataFrame(filas, columns=col_nombres)  
    csv_destino = Path(str(salida_dir)) / f"clase_{clase}.csv"  
    df.to_csv(csv_destino, index=False)  
  
    duracion = time.perf_counter() - inicio  
    if queue is not None:  
        queue.put((clase, duracion)) # devuelve el tiempo al proceso principal
```

cv2.IMREAD_GRAYSCALE lee la imagen directamente en escala de grises, evitando la conversión BGR → RGB → Gris que el código original realizaba en dos pasos. El método flatten() de NumPy opera sobre el bloque contiguo de memoria del arreglo con código compilado en C, eliminando por completo el overhead del intérprete en la extracción de píxeles. Finalmente, en lugar de abrir y cerrar el archivo de salida en cada imagen, todas las filas se acumulan en una lista de arreglos NumPy y se escriben en un solo paso con pandas.DataFrame.to_csv(), reduciendo las operaciones de I/O a una por clase.

2.2 Actividad 2 — Generación de archivos CSV por clase con núcleo dedicado

En el código original, todos los procesos se escribían en el mismo archivo CSV usando el modo append. Esta estrategia introduce contención de escritura: el sistema operativo serializa los accesos a disco, forzando a cada proceso a esperar a que el anterior libere el archivo antes de poder escribir. El efecto neto es que el paralelismo declarado no se traduce en paralelismo real durante la fase de escritura.

En el notebook mantiene la siguiente estructura:

```
# Cola para recoger el tiempo de cada proceso sin necesidad de archivos temporales
cola_tiempos = Queue()

procesos = [
    Process(
        target=procesar_clase,
        args=(clase, DATASET_ROOT, SALIDA_DIR, cola_tiempos),
        name=f"Nucleo-{clase}",
    )
    for clase in CLASES
]

print("Iniciando procesamiento paralelo...")
inicio_paralelo = time.perf_counter()

for p in procesos:
    p.start()

for p in procesos:
    p.join() # espera a que cada proceso termine antes de continuar

tiempo_paralelo = time.perf_counter() - inicio_paralelo

# Recoger los tiempos individuales que cada proceso depositó en la cola
tiempos_por_clase = {}
while not cola_tiempos.empty():
    clase, duracion = cola_tiempos.get()
    tiempos_por_clase[clase] = duracion

print(f"\nProcesamiento paralelo completado en {tiempo_paralelo:.4f} s")
print("\nTiempo por clase (dentro de su propio proceso):")
for clase in CLASES:
    print(f" Clase {clase}: {tiempos_por_clase.get(clase, 0):.4f} s")
```

La solución asigna a cada proceso un archivo de salida exclusivo (clase_A.csv, clase_B.csv, clase_C.csv, clase_D.csv). Los cuatro procesos operan de forma completamente independiente sin ninguna sincronización en escritura. Para comunicar el tiempo de ejecución de cada proceso al proceso principal se utiliza una multiprocessing.Queue, que permite pasar los resultados de forma segura sin recurrir a variables compartidas:

```
cola_tiempos = Queue()
procesos = [
    Process(target=procesar_clase,
            args=(clase, DATASET_ROOT, SALIDA_DIR, cola_tiempos),
```

```

        name=f"Nucleo-{clase}")
    for clase in CLASES
]
for p in procesos: p.start()
for p in procesos: p.join()

```

Una vez que todos los procesos terminan, los cuatro archivos CSV se fusionan en un único dataset_final.csv mediante `pd.concat()`. Esta operación de concatenación es de lectura pura y se ejecuta una sola vez en el proceso principal, sin penalizar el tiempo de procesamiento paralelo.

2.3 Actividad 3 — Comparación secuencial vs. paralelo

Para cuantificar el beneficio del paralelismo se ejecuta el mismo preprocesamiento en dos modalidades. En la modalidad secuencial, cada clase se procesa una tras otra en el mismo hilo de ejecución; el tiempo total es la suma de los tiempos individuales de las cuatro clases. En la modalidad paralela, los cuatro procesos corren simultáneamente; el tiempo total está determinado por la clase que más tarda en procesar.

```

SALIDA_SEQ = SALIDA_DIR / "secuencial"
SALIDA_SEQ.mkdir(exist_ok=True)

print("Iniciando procesamiento secuencial...")
inicio_secuencial = time.perf_counter()

# Sin Process: cada clase espera a que la anterior termine por completo
for clase in CLASES:
    | procesar_clase(clase, DATASET_ROOT, SALIDA_SEQ) # sin queue: solo procesa

tiempo_secuencial = time.perf_counter() - inicio_secuencial

print(f"\nProcesamiento secuencial completado en {tiempo_secuencial:.4f} s")
print(f"Procesamiento paralelo completado en {tiempo_paralelo:.4f} s")
print(f"\nAceleración (speedup) : {tiempo_secuencial / tiempo_paralelo:.2f}x")
print(f"Tiempo ahorrado : {tiempo_secuencial - tiempo_paralelo:.4f} s")

```

La medición se realiza con `time.perf_counter()`, que ofrece la mayor resolución disponible en el sistema operativo:

```

# Modalidad paralela (medida previa):
inicio_paralelo = time.perf_counter()
for p in procesos: p.start()
for p in procesos: p.join()
tiempo_paralelo = time.perf_counter() - inicio_paralelo

```

```
# Modalidad secuencial:
inicio_secuencial = time.perf_counter()
for clase in CLASES:
    procesar_clase(clase, DATASET_ROOT, SALIDA_SEQ)
tiempo_secuencial = time.perf_counter() - inicio_secuencial

speedup = tiempo_secuencial / tiempo_paralelo
```

El speedup teórico máximo bajo la Ley de Amdahl para cuatro procesos y carga perfectamente paralela sería de 4×. El valor empírico observado se aproxima a ese límite cuando el dataset está balanceado, pero cae cuando la distribución de imágenes entre clases es desigual —situación que efectivamente ocurre en este dataset, donde la Clase D cuadruplica en tamaño a la Clase A—. La gráfica de barras generada por el notebook visualiza directamente la diferencia entre ambos tiempos y anota el speedup obtenido.

2.4 Actividad 4 — Tiempo de procesamiento por clase individual

Cada proceso hijo deposita su tiempo de ejecución en la Queue al finalizar. El proceso principal extrae estos valores una vez que todos los procesos terminaron y construye la gráfica de barras con eje secundario. El eje primario muestra el tiempo en segundos; el eje secundario superpone la cantidad de imágenes por clase como línea punteada, evidenciando la correlación entre ambas variables.

El dataset presenta la siguiente distribución de imágenes:

Clase	Imágenes	Tiempo paralelo	Throughput
A	363	Variable (menor carga)	Mayor img/s relativo
B	922	Intermedio	Proporcional a carga
C	990	Intermedio-alto	Proporcional a carga
D	1,388	Mayor (cuello de botella)	Menor img/s relativo

La correlación directa entre el número de imágenes y el tiempo de procesamiento confirma que la rutina optimizada tiene un comportamiento lineal respecto al tamaño de entrada, lo cual es el comportamiento esperado para una operación $O(n)$ vectorizada. Cualquier desviación de la linealidad indicaría

presión sobre la memoria caché o contención en el bus de memoria al acceder a arreglos NumPy de gran tamaño.

Dado que el proceso principal espera a que todos los hijos terminen (`p.join()`), el tiempo total del modo paralelo está acotado inferiormente por el tiempo de la Clase D. Esta observación es central: con cuatro clases y cuatro núcleos, el paralelismo elimina la acumulación de tiempos del modo secuencial, pero no puede eliminar el tiempo del proceso más lento. Un particionado dinámico de la carga —donde los núcleos recogen tareas de una cola compartida en lugar de recibir una clase fija— reduciría este efecto y de esta manera, se acercaría el speedup al límite teórico.

En el notebook, la estructura de código utilizada es la siguiente:

```
imagenes_por_clase = {
    clase: len(list((DATASET_ROOT / clase).glob("*.JPG")))
    for clase in CLASES
}

clases_ord = sorted(tiempos_por_clase.keys())
tiempos_ord = [tiempos_por_clase[c] for c in clases_ord]
imagenes_ord = [imagenes_por_clase[c] for c in clases_ord]

fig, ax1 = plt.subplots(figsize=(8, 4))
ax2 = ax1.twinx()

colores_clase = [ "#81b29a", "#f2cc8f", "#e07a5f", "#3d405b" ]
barras = ax1.bar(clases_ord, tiempos_ord, color=colores_clase, width=0.5, edgecolor="white", label="Tiempo (s)")

ax2.plot(clases_ord, imagenes_ord, color="#264653", marker="o", linewidth=2,
        linestyle="--", label="Imágenes", zorder=5)

for barra, valor in zip(barras, tiempos_ord):
    ax1.text(
        barra.get_x() + barra.get_width() / 2,
        barra.get_height() + 0.05,
        f"{valor:.2f} s",
        ha="center", va="bottom", fontsize=10, fontweight="bold",
    )

ax1.set_xlabel("Clase")
ax1.set_ylabel("Tiempo de procesamiento (s)", color="#3d405b")
ax2.set_ylabel("Número de imágenes", color="#264653")
ax1.set_title("Tiempo de procesamiento por clase\n(ejecución en paralelo – un núcleo por clase)", fontsize=12)
ax1.set_ylim(0, max(tiempos_ord) * 1.3)
ax1.grid(axis="y", alpha=0.3)
ax1.spines[["top"]].set_visible(False)

# Leyendas
lineas1, etiquetas1 = ax1.get_legend_handles_labels()
lineas2, etiquetas2 = ax2.get_legend_handles_labels()
ax1.legend(lineas1 + lineas2, etiquetas1 + etiquetas2, loc="upper left")

plt.tight_layout()
plt.savefig(SALIDA_DIR / "tiempo_por_clase.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gráfica guardada en output/tiempo_por_clase.png")
print("\nResumen por clase:")
```

Resultados

3.1 Optimización de la rutina de procesamiento

La vectorización con NumPy elimina el overhead del intérprete Python en la extracción de píxeles de forma cuantitativa y reproducible. Para una imagen de 16×16, flatten() sustituye 256 evaluaciones de bytecode por una sola llamada a código compilado en C que opera sobre el bloque contiguo de memoria del arreglo. A escala de dataset completo más de 900,000 iteraciones en la versión original, la ganancia evidentemente es sustancial.

La lectura directa en escala de grises con IMREAD_GRAYSCALE elimina adicionalmente dos conversiones de espacio de color. La escritura por lotes reduce el número de operaciones de I/O de N (una por imagen) a 1 (una por clase), aliviando la presión sobre el sistema de archivos.



3.2 Generación de CSV por clase con núcleo dedicado

La arquitectura de escritura sin contención transforma un paralelismo nominal en paralelismo efectivo. En la versión original, la competencia por un archivo compartido forzaba serialización implícita durante la fase de escritura, degradando el beneficio esperado del multiprocesamiento. Al asignar un archivo exclusivo por clase, los cuatro procesos trabajan completamente independientes: no existen esperas, no existe sincronización de escritura, y el sistema operativo puede distribuir la carga de I/O entre los buffers de disco sin arbitraje. La fusión posterior es lineal en el número de archivos y no afecta el tiempo crítico del procesamiento.

3.3 Impacto del paralelismo: secuencial vs. paralelo

La comparación empírica valida la hipótesis central de la práctica: el procesamiento paralelo reduce el tiempo total de ejecución respecto a la modalidad secuencial, con un speedup acotado por la Ley de Amdahl. En el modo secuencial el tiempo es la suma de los tiempos de las cuatro clases; en el modo paralelo el tiempo está dominado por la clase más lenta. Con un dataset de 3,663 imágenes distribuidas en cuatro clases, el paralelismo es efectivo pero no alcanza el speedup teórico de $4\times$ debido al desbalance en la distribución de imágenes entre clases.

3.4 Análisis del tiempo por clase y desbalance de carga

El análisis por clase revela que la Clase D actúa como cuello de botella estructural: con 1,388 imágenes —casi cuatro veces más que la Clase A—, su núcleo asignado tarda significativamente más que los demás, forzando al proceso principal a esperar. Este desbalance es inherente a la estrategia de particionado estático (una clase por núcleo) y no puede mitigarse sin modificar la arquitectura de distribución de tareas. La solución de diseño óptima sería un esquema de work-stealing o una cola compartida de tareas de granularidad fina, donde cada núcleo extrae la siguiente imagen disponible al terminar la anterior. Este refinamiento permitiría acercarse al límite de speedup de $4\times$ independientemente de la distribución de imágenes entre clases, y constituye la extensión natural de esta práctica en un sistema de producción de alto desempeño.

En conjunto, las cuatro actividades demuestran que la optimización de un pipeline de procesamiento de imágenes no es una decisión única sino un proceso

de capas: primero se eliminan las ineficiencias algorítmicas (ciclos anidados), luego las de arquitectura (contención de escritura), luego se mide el impacto real (speedup), y finalmente se diagnostican los límites residuales (desbalance de carga). Cada capa de optimización reduce el tiempo de ejecución de forma independiente; su efecto combinado es mayor que la suma de sus partes.

Conclusiones

3.1 Optimización de la rutina de procesamiento

La vectorización con NumPy elimina el overhead del intérprete Python en la extracción de píxeles de forma cuantitativa y reproducible. Para una imagen de 16×16 , `flatten()` sustituye 256 evaluaciones de bytecode por una sola llamada a código compilado en C que opera sobre el bloque contiguo de memoria del arreglo. A escala de dataset completo —más de 900,000 iteraciones en la versión original— la ganancia es sustancial. La lectura directa en escala de grises con `IMREAD_GRAYSCALE` elimina adicionalmente dos conversiones de espacio de color. La escritura por lotes reduce el número de operaciones de I/O de N (una por imagen) a 1 (una por clase), aliviando la presión sobre el sistema de archivos.

3.2 Generación de CSV por clase con núcleo dedicado

La arquitectura de escritura sin contención transforma un paralelismo nominal en paralelismo efectivo. En la versión original, la competencia por un archivo compartido forzaba serialización implícita durante la fase de escritura, degradando el beneficio esperado del multiprocesamiento. Al asignar un archivo exclusivo por clase, los cuatro procesos trabajan completamente independientes: no existen esperas, no existe sincronización de escritura, y el sistema operativo puede distribuir la carga de I/O entre los buffers de disco sin arbitraje. La fusión posterior es lineal en el número de archivos y no afecta el tiempo crítico del procesamiento.

3.3 Impacto del paralelismo: secuencial vs. paralelo

La comparación empírica valida la hipótesis central de la práctica: el procesamiento paralelo reduce el tiempo total de ejecución respecto a la modalidad secuencial, con un speedup acotado por la Ley de Amdahl. En el modo secuencial el tiempo es la suma de los tiempos de las cuatro clases; en el modo paralelo el tiempo está dominado por la clase más lenta. Con un dataset de 3,663 imágenes distribuidas en cuatro clases, el paralelismo es efectivo pero no alcanza el speedup teórico de $4\times$ debido al desbalance en la distribución de imágenes entre clases.

3.4 Análisis del tiempo por clase y desbalance de carga

El análisis por clase revela que la Clase D actúa como cuello de botella estructural: con 1,388 imágenes —casi cuatro veces más que la Clase A—, su núcleo asignado tarda significativamente más que los demás, forzando al proceso principal a esperar. Este desbalance es inherente a la estrategia de particionado estático (una clase por núcleo) y no puede mitigarse sin modificar la arquitectura de distribución de tareas. La solución de diseño óptima sería un esquema de work-stealing o una cola compartida de tareas de granularidad fina, donde cada núcleo extrae la siguiente imagen disponible al terminar la anterior. Este refinamiento permitiría acercarse al límite de speedup de $4\times$ independientemente de la distribución de imágenes entre clases, y constituye la extensión natural de esta práctica en un sistema de producción de alto desempeño.

En conjunto, las cuatro actividades demuestran que la optimización de un pipeline de procesamiento de imágenes no es una decisión única sino un proceso de capas: primero se eliminan las ineficiencias algorítmicas (ciclos anidados), luego las de arquitectura (contención de escritura), luego se mide el impacto real (speedup), y finalmente se diagnostican los límites residuales (desbalance de carga). Cada capa de optimización reduce el tiempo de ejecución de forma independiente; su efecto combinado es mayor que la suma de sus partes.